Documento Explicativo del Código — SALUNIC

Salud y Bienestar · Nicaragua

Programa: Programación Estructurada — Grupo SALUNIC (6 integrantes)

II Semestre — I Corte Evaluativo

¿Cómo usar este documento?

Este documento fue escrito **para personas que aún no saben programar**. No necesitas ser experto en informática para entenderlo. Está pensado como una **guía de estudio** para que cada integrante del grupo entienda:

1. **Qué es** cada archivo del proyecto y **para qué sirve**.
2. **Cómo está construido** el código, paso a paso y en un lenguaje sencillo.
3. **Cómo funciona** la App gráfica (la parte visual) y el módulo por consola (los menús de texto).
4. **La historia del proyecto**: cómo se fue creando el código a lo largo del semestre.

Recomendaciones para estudiar

- Lee la sección **[Conceptos básicos](#2-conceptos-basicos---lo-que-necesitas-saber-antes-de-ver-codigo)** primero. Ahí están explicadas las palabras que se repiten en todo el código.
- Después lee la sección de la **App gráfica** y luego la del **módulo por consola**.
- Ejecuta el código mientras lees. Verás que todo cobra sentido cuando lo tocas.
- Usa el **glosario** al final si encuentras alguna palabra desconocida.

---

1. ¿Qué es SALUNIC?

SALUNIC es una aplicación de salud y bienestar desarrollada con el lenguaje de programación **Python**, pensada para la realidad de **Nicaragua** (hospitales como el Fernando Vélez Paiz, Hospital Bautista o el Hospital Militar EADB).

El proyecto tiene **dos partes**:

Parte	Qué es	Cómo se ve
App gráfica	Una aplicación con ventanas, botones y colores (diseño tipo celular).	Ventana estilo teléfono de 375×812 píxeles.

Módulo por consola	Un programa que trabaja con menús de texto dentro de la terminal (la "pantalla negra" de comandos).	Menús con números: 1. Gestion de Usuarios, etc.

Ambas partes usan los mismos conceptos de programación, así que aprender una te ayuda a entender la otra.

¿Qué permite hacer?

- **App gráfica:** iniciar sesión, crear cuenta, ver citas médicas, medicamentos, historial médico, estadísticas de salud y editar el perfil.
- **Módulo por consola:** gestionar (registrar, buscar, actualizar, eliminar y listar) usuarios, citas médicas, medicamentos y stock de almacén/inventario. Esto se conoce como operaciones **CRUD** (Crear, Leer, Actualizar y Eliminar).

---

2. Conceptos básicos — lo que necesitas saber antes de ver código

Antes de leer código, es útil conocer estas ideas. **Usa ejemplos de la vida real.**

2.1 ¿Qué es Python?

Python es un **lenguaje de programación**: un idioma que le decimos a la computadora para que haga tareas. Los archivos de Python terminan en **`.py`**.

Para ejecutar un archivo Python desde la terminal se usa:

```
py nombre_del_archivo.py
```

2.2 Variable: una caja con etiqueta

Una **variable** es como una caja donde guardas un dato y le pones una etiqueta.

```
edad = 18
nombre = "Maria"
```

- `edad = 18` → la caja `edad` guarda el número 18.
- `nombre = "Maria"` → la caja `nombre` guarda el texto "Maria".
- Los textos (llamados **strings** o cadenas) siempre van entre comillas `"..."`.

2.3 Tipos de datos más comunes

Tipo	Qué guarda	Ejemplo

int	Números enteros	5, 100, -3
float	Números con decimales	3.14, 0.5
str	Texto (cadena)	"Hola", "+505 86459285"
bool	Verdadero o falso	True, False

2.4 Lista: una fila de casilleros

Una **lista** guarda varios datos en orden, como una fila de casilleros numerados.

Empieza en el **número 0** (el primer casillero es el índice 0).

```
usuarios = ["Carlos", "Ana", "Luis"]
print(usuarios[0])   # Imprime: Carlos
```

2.5 Diccionario: casilleros con nombre

Un **diccionario** guarda datos con una *clave* (nombre) y un *valor*.

```
usuario = {"name": "Maria", "email": "maria@correo.com"}
print(usuario["email"])   # Imprime: maria@correo.com
```

El módulo por consola guarda los datos como **listas de objetos**; la app gráfica

guarda a los usuarios como **diccionarios**.

2.6 Función: una receta reutilizable

Una **función** es como una receta de cocina: tiene un nombre, recibe ingredientes

(**parámetros**), hace algo y puede devolver un resultado.

```
def saludar(nombre):
    return "Hola " + nombre

print(saludar("Ana"))   # Imprime: Hola Ana
```

- `def` → palabra clave para definir (crear) una función.
- `saludar(nombre)` → el nombre de la función y su ingrediente.
- `return` → devuelve el resultado al que la llamó.

2.7 Clase: un molde o plantilla

Una **clase** es un molde para crear objetos con características (**atributos**) y

comportamientos (**métodos** = funciones dentro de la clase).

```
class Usuario:
    def __init__(self, nombre, email):
        self.nombre = nombre
        self.email = email
```

- `__init__` → método especial que se ejecuta al crear un nuevo objeto (el "constructor").
- `self` → representa al objeto mismo; con `self.nombre` guardamos datos *de ese* objeto.

En este proyecto, `entidades.py` del módulo de consola usa clases para definir qué es un Usuario, una Cita, un Medicamento, etc.

2.8 Decorador `@dataclass`: molde sin escribir tanto

`@dataclass` es una "varita mágica" que Python ofrece para crear clases de datos sin tener que escribir el constructor a mano. Se usa en `entidades.py`.

```
from dataclasses import dataclass

@dataclass
class Medicamento:
    id: int
    nombre: str
    dosis: str
```

2.9 Condicional `if`: tomar decisiones

```
hora = 14
if hora < 12:
    print("Buenos dias")
elif hora < 18:
    print("Buenas tardes")
else:
    print("Buenas noches")
```

- `if` → si la condición es verdadera, hace lo primero.
- `elif` → si la primera no se cumple pero esta sí.
- `else` → si ninguna se cumple, hace lo último.
- Los dos puntos `:` indican que viene un bloque de código.
- La **indentación** (espacios al inicio de línea) es obligatoria en Python: marca qué líneas pertenecen a cada bloque.

2.10 Bucle `for`: repetir sobre una lista

```
for u in usuarios:
    print(u)
```

Recorre los casilleros de la lista y ejecuta el bloque una vez por cada elemento.

2.11 Bucle `while`: repetir mientras se cumpla una condición

```
while True:
    print("Esto se repite siempre")
```

El menú del módulo por consola usa `while True` para mantenerse abierto hasta que el usuario elija la opción "Salir" (`0`).

2.12 `import`: traer herramientas de otros archivos

Los proyectos se organizan en varios archivos. `import` sirve para usar en un archivo lo que está definido en otro.

```
import tkinter as tk           # Trae la librería de ventanas
from data import users_data     # Trae el archivo users_data de la carpeta data
import funciones as f           # Trae el archivo funciones y le ponemos un apodo (f)
```

- `as` → crea un apodo (alias). Por ejemplo, `f.registrar_usuario()` es lo mismo que `funciones.registrar_usuario()`.
- **Librería (módulo)** → un archivo con herramientas ya hechas por otros.

2.13 La terminal y el `input()`

En los programas de consola, `input()` muestra un mensaje y espera a que el usuario escriba algo. Lo que escribe se guarda como texto.

```
nombre = input("Escribe tu nombre: ")
```

La app gráfica no usa `input()`: usa **cajas de texto** (`tk.Entry`) y **botones**.

---

3. Estructura general del proyecto (el mapa)

Así se organizan los archivos del proyecto en la raíz `C:\salunic-app`:

```
salunic-app/
│
├── main.py                    ← Arranque de la APP GRÁFICA
├── screens/                   ← Carpeta de las pantallas de la app gráfica
│   ├── splash_screen.py       ← Pantalla de carga con animación
│   ├── inicio_screen.py       ← Pantalla de bienvenida
│   ├── login_screen.py        ← Pantalla de iniciar sesión
│   ├── registro_screen.py     ← Pantalla de crear cuenta
│   ├── home_screen.py         ← Panel de control (inicio del usuario)
│   ├── citas_screen.py        ← Pantalla de citas médicas
│   ├── medicamentos_screen.py ← Pantalla de medicamentos
│   ├── historial_screen.py    ← Pantalla de historial médico
│   ├── estadisticas_screen.py ← Pantalla de estadísticas de salud
│   ├── styles.py              ← Colores, tamaños y herramientas de diseño
│   ├── forms/
│   │   ├── form_registro.py   ← Formulario "Editar Perfil"
│   │   └── form_medicamento.py← Formulario "Agregar Medicamento"
│   └── password_reset/
│       └── screen_2_otp.py    ← Pantalla de código OTP (6 dígitos)
│
├── data/
│   ├── users_data.py          ← Guarda/lee usuarios en un archivo users.json
│   └── users.json             ← Archivo donde quedan guardados los usuarios
```

```
│
├── SALUNIC-Sistema-de-Salud-y-Bienestar/   ← MÓDULO POR CONSOLA
│   ├── main.py                ← Menú principal y submenús
│   ├── entidades.py           ← Clases: Usuario, CitaMedica, Medicamento,
AlmacenMedicamento
│   ├── datos.py               ← Listas vacías + datos precargados de ejemplo
│   └── funciones.py           ← Validaciones y operaciones CRUD
│
├── II Semestre - I Corte/     ← Documentos de la defensa
├── Numeros de Carnet Grupo.txt← Carnets de los 6 integrantes
└── README.md                  ← Descripción general del proyecto
```

Idea clave: *el proyecto separa el* ***diseño visual*** *(la app gráfica) del* ***razonamiento de datos*** *(el módulo de consola). Esto hace el código más ordenado y fácil de mantener.*

---

4. La App Gráfica (tkinter) paso a paso

La app gráfica se construye con **tkinter**, la librería de ventanas que ya viene instalada con Python. No necesitas instalar nada extra para la app gráfica.

4.1 El arranque: `main.py`

Cuando ejecutas `py main.py` (en la raíz del proyecto), Python lee este archivo de arriba hacia abajo. Es el **punto de entrada** de la app.

Paso 1 — Importar las pantallas: al inicio se traen todas las pantallas que la app va a usar:

```
import tkinter as tk
from screens.splash_screen import SplashScreen
from screens.inicio_screen import InicioScreen
from screens.login_screen import LoginScreen
from screens.registro_screen import RegistroScreen
from screens.home_screen import HomeScreen
from screens.citas_screen import CitasScreen
from screens.historial_screen import HistorialScreen
from screens.medicamentos_screen import MedicamentosScreen
from screens.estadisticas_screen import EstadisticasScreen
from screens.forms.form_medicamento import FormMedicamento
from screens.forms.form_registro import FormRegistro
from screens.password_reset.screen_2_otp import Screen2OTP
```

Paso 2 — La clase `SalunicApp`: es el "administrador" de toda la aplicación. Aquí se configuran el título, el tamaño (375×812, como un celular) y se crea el "contenedor"

donde vivirán todas las pantallas:

```
class SalunicApp(tk.Tk):
    def __init__(self):
        tk.Tk.__init__(self)
        self.title("SALUNIC - Salud y Bienestar")
        self.geometry("375x812")
        self.resizable(False, False)
        self.current_user = None
        ...
        self.show_screen("Splash")
```

- `tk.Tk` → la ventana principal.
- `self.current_user = None` → guarda el usuario que inicia sesión. Al principio no hay nadie, por eso es `None`. Cuando alguien inicia sesión, aquí se guarda su información.

Paso 3 — El "cambiador de pantallas" `show_screen`: así navega la app. La primera vez que se pide una pantalla, la **crea** (la guarda en `self.frames`); las siguientes veces solo la **trae al frente** con `tkraise()`:

```
def show_screen(self, name):
    if name in self.frames:
        frame = self.frames[name]
        frame.tkraise()
    else:
        if name == "Splash":
            frame = SplashScreen(self.container, self)
        elif name == "Inicio":
            frame = InicioScreen(self.container, self)
        # ... y así con todas las pantallas ...
        self.frames[name] = frame
        frame.grid(row=0, column=0, sticky="nsew")
        frame.tkraise()
```

Paso 4 — Arrancar la aplicación: la última parte solo se ejecuta si este archivo es el que se está corriendo (no si es importado desde otro):

```
if __name__ == "__main__":
    app = SalunicApp()
    app.mainloop()
```

- `app.mainloop()` → pone la app en "modo espera de eventos": se queda funcionando esperando que el usuario haga clic, escriba, etc.

4.2 El flujo de pantallas (el camino del usuario)

¿Te parece un formulario de navegación? Esto es lo que ocurre en tiempo real:

```
[1] Splash (carga animada, ~1.5 segundos)
        │
        ▼
[2] Inicio (bienvenida)
```

```
        │  botón "COMENZAR" / "INICIAR SESION"
        ▼
[3] Login (correo + contraseña)
        │  (se verifica el usuario)
        ▼
[4] Home (panel de control) ──────────► Citas / Medicamentos / Historial / Perfil
        │
        ├── Citas → (botón volver) → Home
        ├── Medicamentos → Agregar Medicamento (FormMedicamento) → Medicamentos
        ├── Historial → (botón volver) → Home
        ├── Estadísticas → (botón volver) → Home
        └── Mi Perfil (FormRegistro) → Guardar → Home
```

Además, desde **Login** se puede ir a:

- **Registro** (crear cuenta nueva) → al crearla va directo a Home.
- **Screen2OTP** (restablecer contraseña con código de 6 dígitos) → si es correcto, vuelve al Login.

4.3 `styles.py`: la identidad visual del proyecto

Este archivo concentra los **colores, tamaños y formas** de la app para que todos los diseños sean consistentes. Es como la "paleta de pinturas" del proyecto.

```
VERDE = "#2ECC71"
FUCSIA = "#E91E8C"
AZUL = "#1565C0"
CELESTE = "#4FC3F7"
AMARILLO = "#F9A825"
AZUL_NOCHE = "#0A2342"
...
WIDTH = 375
HEIGHT = 812
```

- Los colores se escriben en formato **hexadecimal** (`#RRGGBB`), que es la forma en que tkinter entiende los colores.
- `GRADIENT_COLORS` define los colores que se combinan para crear el **fondo degradado** que se ve en todas las pantallas.
- `create_rounded_rect(canvas, x1, y1, x2, y2, r, **kwargs)` dibuja rectángulos con esquinas redondeadas (como las "cards" que se ven en el diseño). Internamente usa `canvas.create_polygon(..., smooth=True)` para suavizar las esquinas.

> 💡 ***Detalle curioso:*** *el degradado de cada pantalla no es una imagen: se dibuja línea por línea. Por eso en muchas pantallas ves un código casi idéntico que recorre de arriba a abajo calculando el color de cada franja. Es un patrón repetido a*

propósito para que cada pantalla dibuje su propio fondo.

4.4 La pantalla Splash (de carga)

`SplashScreen` es lo primero que ves. Dibuja el logo de SALUNIC y dos círculos que giran alrededor de la letra "S".

La animación funciona con un truco de tkinter llamado `after()`:

```
self.after(50, self._animate)   # "ejecuta _animate dentro de 50 milisegundos"
```

Cada vez que se ejecuta `_animate`:

5. Aumenta el ángulo: `self.angle += 4`.
6. Recalcula la posición de los círculos usando **matemáticas** (seno y coseno) para que se muevan en círculo.
7. Se vuelve a programar a sí misma con `after(50, self._animate)` (¡bucle!).

Cuando el ángulo llega a 360 (una vuelta completa), espera 1.5 segundos y cambia a la pantalla de Inicio:

```
if self.angle >= 360:
    self.angle = 0
    self.after(1500, lambda: self.controller.show_screen("Inicio"))
    return
```

***Concepto:** Lambda crea una función pequeña "sobre la marcha", sin nombre. Aquí sirve para decir "después de 1.5 segundos, cambia a la pantalla Inicio".*

4.5 Pantalla Inicio (bienvenida)

`InicioScreen` muestra el mensaje de bienvenida y tres botones. Un detalle: los botones no se hacen con el widget `Button` de tkinter, sino con `Label` (etiqueta) sobre un rectángulo redondeado, para tener total control del diseño:

```
btn1 = tk.Label(self, text="COMENZAR", font=("Nunito", 14, "bold"), fg=BLANCO,
bg=VERDE, cursor="hand2")
btn1.place(...)
btn1.bind("<Button-1>", lambda e: self.controller.show_screen("Login"))
```

- `bind("<Button-1>", ...)` → dice: "cuando el usuario haga **clic izquierdo**, haz esto".
- El clic ejecuta `self.controller.show_screen("Login")`. El `controller` es la misma instancia de `SalunicApp`, así la pantalla pide al administrador que cambie de vista.

4.6 Pantalla Login (iniciar sesión)

`LoginScreen` tiene dos cajas (`tk.Entry`): email y contraseña. Además, viene **pre-rellenada** para probar la app rápidamente:

```
self.email_entry.insert(0, "usuario@salunic.com")
self.password_entry.insert(0, "123456")
```

Cuando el usuario hace clic en "INICIAR SESION", se ejecuta `_login()`:

8. Lee lo que hay en las cajas.
9. Verifica que no estén vacías y que el email tenga `@` y un punto.
10. Llama a `users_data.find_user(email, password)` (lee el archivo `users.json`).
11. Si no existe, ofrece un usuario de demostración con `usuario@salunic.com` / `123456` (así siempre puedes entrar).
12. Guarda al usuario en `self.controller.current_user` y muestra el Home.

```
self.controller.current_user = user
self.controller.show_screen("Home")
```

> ***Punto importante para estudiar:*** *el Login valida los datos antes de dejar pasar.*
>
> *Validar siempre es un paso extra de seguridad y de orden.*

4.7 Pantalla Registro (crear cuenta)

`RegistroScreen` pide: nombre, email, contraseña y confirmar contraseña. Al hacer clic en "CREAR CUENTA", `_register()`:

13. Verifica que **ningún campo esté vacío** (`if not all([...])`).
14. Verifica que la contraseña y su confirmación sean iguales.
15. Verifica que la contraseña tenga al menos 6 caracteres.
16. Evita registros duplicados con `users_data.email_exists(email)`.
17. Guarda el nuevo usuario con `users_data.add_user(name, email, pw)`.
18. Inicia sesión automáticamente (guarda el usuario en `current_user`) y va al Home.

> ***Nota:*** *la app gráfica tiene reglas distintas a las del módulo de consola*
>
> *(ahí la contraseña mínima es de 4 caracteres). Son dos programas independientes.*

4.8 Home (panel de control)

`HomeScreen` es la pantalla principal después del login. Tiene varias partes:

- **Saludo según la hora** (`_greeting()`): usa `datetime.now().hour` para decidir si dice "Buenos días", "Buenas tardes" o "Buenas noches".
- **Iniciales del usuario** (`_initials()`): toma la primera letra de las dos primeras palabras del nombre.
- **Datos vitales** (temperatura, pulso, citas, medicamentos).
- **Tarjetas de servicios** clicables: Citas, Medicamentos, Historial y Mi Perfil.

```
serv = [
    ("#1565C0", "Citas Medicas\n2 proximas", "", "Citas"),
    ("#0A4D2E", "Medicamentos\n2 tomas hoy", "", "Medicamentos"),
```

```
    ...
]
for (color, text, icon, dest), (x, y) in zip(serv, positions):
    ...
    l.bind("<Button-1>", lambda e, d=dest: self.controller.show_screen(d))
```

> ***Detalle de programación:*** *dentro del `bind` se usa `lambda e, d=dest: ...`. Ese `d=dest` "congela" el valor del destino en cada vuelta del bucle. Sin ese truco, los cuatro botones terminarían apuntando al mismo destino (el último). Es un detalle clásico del Python llamado "cierre de variable en bucle".*

- **Lista de próximas citas** dibujadas como tarjetas.
- **Barra de pie** (footer) con navegación: Inicio, Estadísticas, Perfil y Salir.

El Home tiene **scroll** (desplazamiento): el contenido se dibuja en un `tk.Canvas` interior con una barra de desplazamiento vertical.

4.9 Pantallas de contenido (Citas, Medicamentos, Historial, Estadísticas)

Estas cuatro pantallas son muy parecidas entre sí, y son las más simples:

```
citas = [
    ("15", "JUN", "Dr. Carlos Ruiz", "Medicina General", "10:30 AM", "Hospital
Militar EADB", VERDE),
    ...
]
for dia, mes, doctor, espec, hora, lugar, color in citas:
    # dibuja una tarjeta redondeada con la información
```

Patrón que repiten las cuatro:

19. Dibujan el fondo degradado.
20. Dibujan la barra superior (nav) con botón "atrás" (←) que vuelve al Home.
21. Dibujan el título.
22. Recorren una lista de datos con `for` y dibujan una tarjeta (`create_rounded_rect`) por cada elemento.
23. Dibujan el botón "VOLVER AL INICIO" al final.

> ***Idea de estudio:*** *nota cómo cambiar de pantalla es solo cambiar los datos de la lista y los textos. La estructura del código es la misma. Si entiendes una de estas cuatro, las entiendes todas.*

4.10 Formulario "Agregar Medicamento" (`FormMedicamento`)

Es una pantalla con desplazamiento que recoge la información de un nuevo medicamento:

- Nombre, dosis y presentación (cajas de texto).

- **Frecuencia**: 4 botones (1X DIA, 2X DIA, 3X DIA, C/8H) que se marcan/resaltan al pulsarlos (`_toggle_frec`).
- Horarios (2 cajas).
- Instrucciones.
- Casilla "Notificación push" (checkbox con `tk.BooleanVar`).

Los campos usan **placeholders**: muestran un texto gris de ejemplo (ej: "500mg") y lo borran cuando el usuario hace clic dentro (`<FocusIn>`), y lo restauran si el usuario lo deja vacío y sale (`<FocusOut>`). Eso lo hacen `_fi` y `_fo`.

Al pulsar "GUARDAR MEDICAMENTO", `_guardar()` valida que el nombre, la dosis y al menos un horario estén completos, y muestra un mensaje de confirmación.

4.11 Formulario "Editar Perfil" (`FormRegistro`)

Sirve para editar la información personal. Tiene dos detalles especiales:

24. **Pre-rellena los campos** con los datos del usuario actual (`_prefill()`), usando la información guardada en `current_user`.
25. Al guardar (`_guardar()`), valida el nombre, el correo y que se acepten los términos, construye el nombre completo, actualiza `current_user` y llama a `users_data.update_user(...)` para guardar los cambios en `users.json`.

4.12 Pantalla OTP (`Screen2OTP`)

Permite "restablecer la contraseña" ingresando un código de **6 dígitos**. Lo interesante es la lógica de las cajas:

- Tiene 6 `tk.Entry`, cada una de un solo dígito.
- Al escribir un dígito, el foco salta a la siguiente caja.
- Con `BackSpace`, vuelve a la anterior y borra.

Eso lo hace `_on_key`:

```
def _on_key(self, event, idx):
    if event.keysym == "BackSpace" and idx > 0:
        self.otp_entries[idx - 1].focus()
        self.otp_entries[idx - 1].delete(0, "end")
    elif event.char.isdigit() and idx < 5:
        self.otp_entries[idx + 1].focus()
```

Al verificar (`_verificar()`), junta los 6 dígitos en un solo texto y compara con el código `"123456"`. Si coincide, muestra éxito y vuelve al Login.

4.13 Cómo se guardan los usuarios de la app gráfica (`users_data.py`)

La app gráfica guarda a los usuarios en un **archivo real** llamado `users.json` para que no se pierdan al cerrar la app. Lo hace con formato **JSON** (una forma de escribir datos que Python entiende fácil).

`data/users_data.py` tiene 5 funciones pequeñas:

Función	Qué hace
_load()	Lee el archivo users.json y devuelve su contenido (si no existe, devuelve []).
_save(users)	Escribe la lista de usuarios en users.json.
add_user(name, email, password)	Agrega un usuario nuevo y guarda el archivo.
find_user(email, password)	Busca un usuario con ese correo y contraseña; devuelve el usuario o None.
update_user(email_actual, name, email_nuevo)	Actualiza nombre/correo de un usuario existente.
email_exists(email)	Devuelve True/False si ese correo ya está registrado.

```
DB_PATH = os.path.join(os.path.dirname(__file__), "users.json")

def _load():
    if not os.path.exists(DB_PATH):
        return []
    with open(DB_PATH, "r", encoding="utf-8") as f:
        return json.load(f)
```

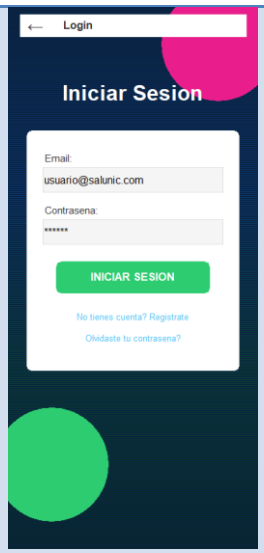
Concepto: *`with open(...) as f:` abre el archivo, lo usa dentro del bloque y lo cierra solo. Es la forma segura y recomendada de trabajar con archivos en Python.*

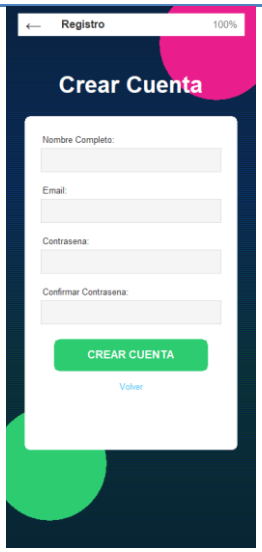
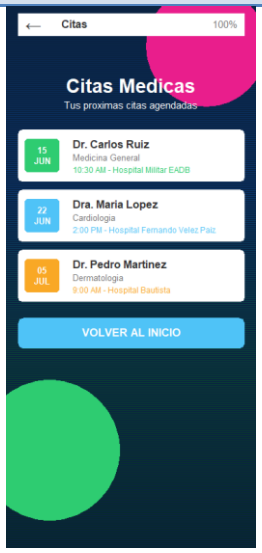
Diferencia con el módulo por consola: *el módulo de consola guarda los datos solo en la memoria (mientras el programa está corriendo). La app gráfica los guarda en un archivo. ¿Por qué? Porque fueron diseñados con propósitos distintos y en momentos distintos del semestre.*

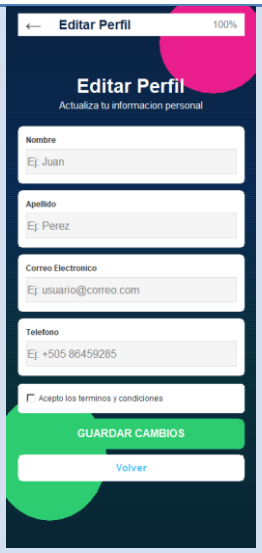
4.14 Galería de pantallas (así se ve la app)

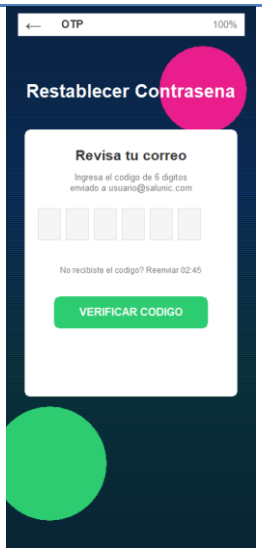
Estas son las capturas reales de la app. Usa esta galería para identificar cada pantalla por su nombre y para practicar *antes* de abrir el código. Así, cuando leas un archivo (`splash_screen.py`, `login_screen.py`, etc.), ya sabrás exactamente cómo se ve esa pantalla.

N.º	Captura	Pantalla	Archivo donde está el código

1		Pantalla de carga con el logo animado	screens/splash_screen.py
2		Bienvenida con botones	screens/inicio_screen.py
3		Iniciar sesión	screens/login_screen.py

4		Crear cuenta	screens/registro_screen.py
5		Panel de control	screens/home_screen.py
6		Citas médicas	screens/citas_screen.py

7		Historial médico	screens/historial_screen.py
8		Agregar medicamento	screens/forms/form_medicamento.py
9		Editar perfil	screens/forms/form_registro.py

10		Restablecer contraseña (OTP)	screens/password_reset/screen_2_otp.py

La **ruta al código** que aparece en la tabla es la carpeta donde vive cada pantalla.

Por ejemplo, para abrir la pantalla del login buscas el archivo `login_screen.py` dentro de la carpeta `screens`.

> 🔍 ***Ejercicio para estudiar:*** *abre la app, navega hasta la pantalla de la captura y luego abre su archivo de código. Compáralo con lo explicado en las secciones 4.4 a 4.12. La pantalla real y el código van de la mano.*

---

5. El módulo "SALUNIC — Sistema de Salud y Bienestar" (por consola) paso a paso

Esta carpeta (`SALUNIC-Sistema-de-Salud-y-Bienestar`) contiene la versión del programa que trabaja con **menús de texto** en la terminal. Fue el primer prototipo funcional del proyecto, desarrollado en las primeras etapas.

Para ejecutarlo:

```
cd SALUNIC-Sistema-de-Salud-y-Bienestar
py main.py
```

Usa una arquitectura ordenada de **4 archivos**:

```
SALUNIC-Sistema-de-Salud-y-Bienestar/
├── entidades.py    ← Define los "moldes" (clases) de los datos
├── datos.py        ← Las listas + datos de ejemplo precargados
├── funciones.py    ← Las operaciones (CRUD) y validaciones
└── main.py         ← Los menús (la parte que el usuario ve)
```

5.1 `entidades.py`: los moldes de los datos

Define **qué es** cada cosa del sistema usando clases con `@dataclass`:

```
from dataclasses import dataclass

@dataclass
class Usuario:
    id: int
    nombre: str
    email: str
    password: str
    telefono: str = "+505 00000000"

@dataclass
class CitaMedica:
    id: int
    paciente_id: int
    medico_id: int
    especialidad: str
    fecha: str        # formato DD/MM/AAAA
    hora: str         # formato HH:MM
    lugar: str = "Hospital"

@dataclass
class Medicamento:
    id: int
    nombre: str
    dosis: str
    frecuencia: str
    presentacion: str = "Tabletas"

@dataclass
class AlmacenMedicamento:
    id: int
    medicamento_id: int
    cantidad: int
    fecha_vencimiento: str  # formato DD/MM/AAAA
```

Observa la **relación entre entidades** (esto se conoce como un modelo de datos y lo ponen en práctica las "Data Clases"):

- `CitaMedica.paciente_id` y `CitaMedica.medico_id` → hacen referencia al `id` de un `Usuario`.
- `AlmacenMedicamento.medicamento_id` → hace referencia al `id` de un `Medicamento`.

> ***Concepto:*** *el `id` es el número de identificación único de cada registro. Gracias a los `id` podemos saber qué paciente tiene qué cita, aunque estén en listas distintas.*

5.2 `datos.py`: las listas y los datos precargados

Primero crea las **listas vacías** que guardarán los datos mientras el programa corre:

```
usuarios = []
citas = []
medicamentos = []
almacen = []
```

Después define **datos de ejemplo** para que el programa ya venga "poblado" cuando se ejecuta:

- `usuarios_iniciales`: 5 usuarios (3 pacientes y 2 médicos).
- `citas_iniciales`: 3 citas en hospitales de Nicaragua.
- `medicamentos_iniciales`: 12 medicamentos reales de la Lista Básica de Medicamentos Esenciales del MINSA.
- `inventario_inicial`: 12 registros de stock con fechas de vencimiento.

```
usuarios_iniciales = [
    Usuario(id=1, nombre="Carlos Ruiz", email="carlos@correo.com",
password="1234", telefono="+505 86459285"),
    Usuario(id=2, nombre="Ana Lopez", email="ana@correo.com", password="1234",
telefono="+505 87351234"),
    ...
]

medicamentos_iniciales = [
    Medicamento(id=1, nombre="Paracetamol", dosis="500mg", frecuencia="Cada 8
horas", presentacion="Tabletas"),
    ...
]
```

Idea de estudio: *los datos de ejemplo dan contexto realista al programa y sirven para probar las funcionalidades sin tener que escribir todo a mano cada vez.*

5.3 `funciones.py`: las operaciones y validaciones

Es el archivo más importante del módulo. Contiene **dos grupos de funciones**:

Grupo 1: Validaciones (funciones que revisan datos)

Cada una recibe texto y lanza un error (`ValueError`) si el dato no es válido:

Función	Verifica
validar_nombre	Que no esté vacío y solo tenga letras y espacios. Usa re.match (expresiones regulares).
validar_email	Que tenga el formato usuario@correo.com.
validar_password	Que tenga al menos 4 caracteres.

validar_fecha	Que sea DD/MM/AAAA con una fecha real (datetime.strptime).
validar_hora	Que sea HH:MM.
validar_celular	Que tenga dígitos (8 a 15) y opcionalmente +.
validar_cantidad	Que sea un número entero positivo.
validar_descripcion	Que no esté vacío.

Un ejemplo:

```
def validar_fecha(fecha):
    try:
        datetime.strptime(fecha, "%d/%m/%Y")
    except ValueError:
        raise ValueError("La fecha debe tener formato DD/MM/AAAA con una fecha
real.")
    return fecha
```

- `strptime` → intenta interpretar el texto como fecha con el formato dado.
- `try / except` → "intenta esto; si falla con este error, haz aquello".

También hay una función **`pedir_campo`** que junta ambos: pide el dato con `input()` y lo valida, y si falla **vuelve a preguntar** (bucle `while True`) hasta que el dato sea válido:

```
def pedir_campo(etiqueta, validador):
    while True:
        try:
            valor = input(etiqueta)
            return validador(valor)
        except ValueError as e:
            print(f"  [Error] {e}")
```

Grupo 2: CRUD de cada entidad

Cada entidad tiene sus **5 operaciones CRUD** (las recuerdas: Crear, Leer, Actualizar, Eliminar + un contador). Por ejemplo, para **Usuarios**:

- `registrar_usuario()` → pide los datos con `pedir_campo`, verifica que el email no exista ya, calcula el nuevo `id` (`max([...]) + 1`) y agrega el `Usuario` a la lista.
- `buscar_usuario(email)` → recorre la lista y devuelve el usuario que coincida.
- `listar_usuarios()` → imprime todos.
- `actualizar_usuario(email)` → encuentra al usuario y permite cambiar nombre y teléfono.
- `eliminar_usuario(email)` → encuentra al usuario y lo quita de la lista.
- `contar_usuarios()` → devuelve `len(usuarios)` (cuántos hay).

Cada una es una función pequeña y con un solo propósito:

```
def buscar_usuario(email):
    for u in usuarios:
        if u.email == email:
            return u
    return None
```

> ***Concepto importante qué debes dominar:*** *el patrón de* ***buscar antes de actuar****.*
>
> *Para actualizar o eliminar un registro primero se busca; si no se encuentra, se muestra un mensaje y no se hace nada. Esto evita errores.*

Los medicamentos, citas y stock usan el **mismo patrón**, solo cambia el dato por el que se busca (nombre, id del médico, id del medicamento) y la lista sobre la que se trabaja.

Un detalle: en `listar_stock` el programa necesita mostrar el **nombre del medicamento** (aunque el stock solo guarda su `id`), así que hace un pequeño recorrido para encontrarlo:

```
nombre_med = "Desconocido"
for m in medicamentos:
    if m.id == a.medicamento_id:
        nombre_med = m.nombre
        break
```

> *Esto es un ejemplo real de cómo se* ***relacionan las entidades*** *en el código: el stock "sabe" cuál medicamento es por el `id`, y el programa busca el nombre.*

5.4 `main.py`: los menús (la parte que el usuario ve)

Este archivo muestra los menús y decide qué función ejecutar según la opción elegida.

El menú principal:

```
============================================================
SALUNIC - Sistema de Salud y Bienestar
============================================================
SELECCIONE UN MODULO:
1. Gestion de Usuarios
2. Gestion de Citas Medicas
3. Gestion de Medicamentos
4. Gestion de Almacen / Inventario
------------------------------------------------------------
0. Salir
```

Paso 1 — Pedir la opción con validación (`pedir_opcion`):

```
def pedir_opcion():
    while True:
        try:
```

```
            opc = input("Seleccione una opcion: ").strip()
            if not opc.isdigit():
                raise ValueError("Debe ingresar un numero.")
            return int(opc)
        except ValueError as e:
            print(f"  [Error] {e}")
```

- `strip()` quita espacios; `isdigit()` comprueba que sea número. Si no es número, **vuelve a preguntar** (por eso el `while True`).

Paso 2 — Los submenús: cada módulo tiene su propio submenú. Por ejemplo, el de usuarios:

```
def submenu_usuarios():
    while True:
        cabecera_modulo("GESTION DE USUARIOS")
        print("1. Registrar un usuario")
        print("2. Buscar un usuario")
        print("3. Actualizar un usuario")
        print("4. Eliminar un usuario")
        print("5. Contar usuarios")
        print("6. Listar todos los usuarios")
        print(SUB)
        print("0. Volver al menu principal")

        opc = pedir_opcion()

        if opc == 0:
            return
        elif opc == 1:
            f.registrar_usuario()
        elif opc == 2:
            ...
```

- Cada opción del menú llama a la función correspondiente de `funciones.py`: `f.registrar_usuario()`, `f.buscar_usuario(...)`, etc.
- La opción `0` hace `return`, que **sale del submenú** y vuelve al menú principal.

Las otras opciones no rompen el `while True`, así el submenú permanece abierto.

Paso 3 — La función principal `main()`:

```
def main():
    f.cargar_datos_iniciales()

    while True:
        mostrar_menu_principal()
        opc = pedir_opcion()

        if opc == 0:
            print("\nGracias por usar SALUNIC. Hasta pronto!")
```

```
            break
        elif opc == 1:
            submenu_usuarios()
        elif opc == 2:
            submenu_citas()
        elif opc == 3:
            submenu_medicamentos()
        elif opc == 4:
            submenu_almacen()
        else:
            print("  [Error] Opcion invalida.")
```

26. Primero **carga los datos de ejemplo** (`cargar_datos_iniciales()`).
27. Luego entra en un bucle infinito (`while True`): muestra el menú, pide una opción y ejecuta lo correspondiente. Solo sale cuando se elige `0` (`break`).

Paso 4 — El arranque:

```
if __name__ == "__main__":
    main()
```

> ***Comparación:*** *ves que el módulo de consola y la app gráfica tienen el mismo patrón final (`if __name__ == "__main__":`). Además, en el módulo de consola el menú se resuelve con una cadena de `if / elif`; en la app gráfica la navegación se resuelve también con `if / elif`, pero dentro del método `show_screen`. Son dos caras de la misma moneda.*

5.5 Recorrido de ejemplo: registrar un usuario, paso a paso

Imagina que eliges `1 → 1` en el menú (Módulo Usuarios → Registrar un usuario):

28. `submenu_usuarios()` detecta `opc == 1` y llama a `f.registrar_usuario()`.
29. `registrar_usuario()` pide: Nombre completo → valida con `validar_nombre`.
30. Pide: Email → valida con `validar_email` y **verifica que no exista ya**.
31. Pide: Contraseña → valida con `validar_password` (mínimo 4 caracteres).
32. Pide: Teléfono → valida con `validar_celular`.
33. Calcula el nuevo id: `max([u.id for u in usuarios], default=0) + 1`. Si hay ids 1,2,3, el nuevo es 4.
34. Crea el objeto `Usuario(...)` y lo agrega a la lista `usuarios` con `append`.
35. Muestra: `Usuario registrado con exito. ID: 4`.
36. El programa regresa al submenú (sigue el `while True`) esperando otra acción.

> ***Concepto:*** *`max([u.id for u in usuarios], default=0) + 1` se lee así: "toma los ids de todos los usuarios (por eso `for u in usuarios`), busca el más grande, y si no hay ninguno usa 0; luego súmale 1". Así los ids **nunca se repiten**.*

6. Historial del código: cómo se creó paso a paso

Esta sección cuenta la **historia real del proyecto** a partir del historial de commits de git (el "diario de cambios" del repositorio). Leerla te ayuda a entender *por qué* el proyecto está organizado como está.

> ***Concepto:*** *un* ***commit*** *es una "foto" del proyecto en un momento dado. Cada commit guarda los cambios hechos con su mensaje. Git es la herramienta que registra esta historia.*

Fase 1 — Los primeros pasos (inicio del I Semestre)

Commit	Qué se hizo
61ff4a4	Commit inicial: se crea el repositorio para versionar el proyecto.
d14362b	Se agrega el primer archivo principal de la aplicación SALUNIC.
28c6dd0	Se crea la pantalla Home (el panel de control).
256edae	Se crea la pantalla de verificación OTP (código de 6 dígitos).
682c5ce	Se crea el formulario de registro.
60573a7	Se crea el formulario de medicamento.

Fase 2 — Diseño profesional (inspirado en Figma)

Commit	Qué se hizo
929b91a	Se actualiza main.py para mostrar el Home con un diseño profesional basado en un prototipo de Figma.
29fcf5f	Fix urgente aplicado: se reemplazan todos los colores RGBA por HEX porque tkinter no soporta colores con transparencia. ¡Esto soluciona los colores en toda la app!
36b0f0a	Fix: se corrigen tamaños de fuente con decimales (ej: 12.5) y se remueven caracteres especiales que tkinter no aceptaba.

> ***Lección de la Fase 2:*** *cuando se diseña con una herramienta visual (Figma) y luego se pasa a código, surgen diferencias técnicas (formatos de color, tamaños de fuente). Corregirlas lleva a los llamados fixes. ¡Es normal!*

Fase 3 — Las pantallas completas (base de la app actual)

Commit	Qué se hizo
116bde2	Pantalla Splash con círculos animados.
32de62e	Pantalla Inicio con botones funcionales.
692b15c	Pantalla Login con validación.
da718e3	Pantalla Registro con formulario completo.
0223a11	main.py con navegación entre pantallas (el método show_screen).
21c762a	Se crea la carpeta screens como paquete de Python (con su __init__.py).
8aff0c9	Se crean todas las pantallas (Inicio, Login, Registro, Home).
b647220	Home actualizado con interfaz completa: datos vitales, servicios y citas.

Concepto: *un* ***paquete*** *es una carpeta con archivos* `.py` *que se pueden importar como módulo (*`from screens.splash_screen import ...`*). El archivo* `__init__.py` *marca que la carpeta es un paquete.*

Fase 4 — Documentación y limpieza (final del I Semestre)

Commit	Qué se hizo
660ffe3	Versión completa y real de SALUNIC con todas las pantallas y documentación.
6188636	Se simplifica requirements.txt.
77638ff	Se mejora el README.md.
8b5cb98	Se mejora .gitignore.
8ab2b74	Se limpia el proyecto: se organiza y se documenta el prototipo de consola del 1er corte.
f75f38a	Limpieza de código: se eliminan variables, imports y objetos muertos (código que no se usa).

Lección: *"limpiar el código" (quitar lo que no se usa) es una práctica de buenos programadores. Deja el proyecto más ligero y fácil de entender.*

Fase 5 — El módulo de consola toma forma

Commit	Qué se hizo
bf77c77	Se renombra la carpeta del prototipo a SALUNIC-

	Sistema-de-Salud-y-Bienestar.
6553dfe	Se mejora el menú: submenús por módulo con retorno al menú principal. (Antes, quizás, todo estaba en un solo menú largo; ahora cada módulo tiene sus opciones.)
d5bfbe9	Se quita el texto "Grupo 8" del título del menú (ya no aplicaba).
6fa16c1	Se precargan usuarios y citas médicas de ejemplo.
430e8a6	Se actualiza el README para reflejar menú principal + submenús y datos precargados.

> ***Lección:*** *renombrar carpetas, reorganizar menús y actualizar la documentación cuando algo cambia son parte del mantenimiento normal de un proyecto.*

Fase 6 — II Semestre: documentos y grupo nuevo

Commit	Qué se hizo
fa361c8	Se agregan los documentos del I corte del II semestre, los carnets del grupo y se limpia el repo.
9e797e3	Se armonizan los integrantes: se deja solo al grupo nuevo de 6 integrantes en toda la documentación.

Resumen visual de la evolución

```
Commit inicial → Pantallas individuales → Fixes de diseño → Todas las
pantallas unidas con navegación → Documentación → Prototipo de consola
con submenús y datos precargados → Documentos del II semestre
```

---

7. Diferencias entre la App gráfica y el módulo por consola (comparativo de estudio)

Aspecto	App gráfica (tkinter)	Módulo por consola
Cómo se ejecuta	py main.py (en la raíz)	cd SALUNIC-Sistema-de-Salud-y-Bienestar + py main.py
Interfaz	Ventanas, botones, colores, animación	Menús de texto con números
Entrada de datos	Cajas de texto (tk.Entry) y botones	input() en la terminal
Dónde guarda los datos	En el archivo users.json (persistente)	Solo en la memoria mientras corre
Concepto principal	Navegación entre pantallas (show_screen)	Menús con while True e if/elif

Organización	Una clase por pantalla	Funciones separadas por tema
Objetivo	Mostrar una experiencia de usuario visual	Demostrar la lógica de datos (CRUD)

---

8. Glosario rápido

Término	Significado sencillo
Python	Lenguaje de programación con el que está hecho el proyecto.
.py	Extensión de los archivos de código Python.
tkinter	Librería de Python para crear ventanas (interfaz gráfica).
Variable	Una caja con etiqueta que guarda un dato.
Lista (list)	Varios datos en orden (casilleros numerados desde 0).
Diccionario (dict)	Datos guardados por nombre de clave y valor.
Booleano (bool)	True (verdadero) o False (falso).
Función	Una receta reutilizable con nombre (hecha con def).
Parámetro	El "ingrediente" que recibe una función.
return	Devuelve el resultado de una función.
Clase	Molde para crear objetos con atributos y métodos.
@dataclass	Decorador que crea clases de datos de forma corta.
Atributo	Una característica de un objeto (ej: usuario.email).
Método	Una función que pertenece a una clase.
if / elif / else	Toma de decisiones por condiciones.
Bucle for	Repite un bloque por cada elemento de una lista.
Bucle while	Repite mientras una condición sea verdadera.
break	Sale del bucle inmediatamente.
import	Trae herramientas de otro archivo o librería.
lambda	Función pequeña sin nombre, creada "al vuelo".
self	Palabra que representa "este mismo objeto" dentro de una clase.
Método __init__	Lo que se ejecuta al crear un objeto (constructor).
try / except	Intenta algo y, si falla, maneja el error sin romper el programa.
raise	Envía un error a propósito.

Expresión regular (re)	Patrón para buscar/comprobar texto (ej: formato de email).
strptime	Interpreta un texto como fecha/hora con un formato dado.
CRUD	Operaciones básicas: Crear, Leer/Listar, Actualizar, Eliminar (y contar).
JSON	Formato de texto para guardar datos (se usa en users.json).
.gitignore	Lista de archivos/carpetas que git debe ignorar.
Commit	"Foto" guardada de los cambios del proyecto en git.
Paquete	Carpeta con módulos .py importables (tiene __init__.py).
Placeholder	Texto gris de ejemplo dentro de una caja que se borra al escribir.
Widget	Un elemento visual de la interfaz (botón, caja, etiqueta...).
canvas	Lienzo de tkinter donde se puede dibujar libremente.
Gradiente	Degradado suave entre dos o más colores.

---

9. Preguntas de autoevaluación

Úsalas para repasar individualmente o en los repasos grupales:

37. ¿Qué comando ejecuta la app gráfica? ¿Y el módulo por consola?
38. ¿Qué hace `self.controller.show_screen("Home")`?
39. ¿Por qué la primera vez que se muestra una pantalla se "crea" y al volver solo se trae al frente?
40. ¿Qué diferencia hay entre una lista y un diccionario? Da un ejemplo de cada uno usado en el proyecto.
41. Explica con tus palabras qué es `@dataclass` y dónde se usa en el proyecto.
42. En el Login, ¿qué validaciones se hacen antes de dejar entrar al usuario?
43. ¿Cómo se calcula el nuevo `id` al registrar un usuario en el módulo de consola?
44. ¿Por qué `validar_fecha` usa `try / except`?
45. ¿Qué hace `pedir_campo` si el usuario ingresa un dato inválido?
46. Nombra las 5 operaciones de un CRUD y la función que las implementa para usuarios.
47. ¿Qué guarda la app gráfica en `users.json` y para qué sirve `email_exists`?
48. ¿Qué patrón se repite en las pantallas de Citas, Medicamentos, Historial y Estadísticas?
49. ¿Con qué librea de Python se dibujan las tarjetas redondeadas? (Pista: está en

`styles.py` y usa `create_polygon` con `smooth=True`).

50. ¿Qué papel juega el `id` médico/medicamento en las relaciones entre entidades?
51. ¿Por qué la Fase 2 del historial tuvo "fixes" de colores y fuentes?

---

10. Mini-tutorial de Git (para los 6 integrantes)

Git es la herramienta que **lleva el diario de cambios** del proyecto. GitHub es la **"nube"** donde está guardado ese proyecto en internet para que todos puedan verlo y trabajar juntos.

> ***Idea clave:*** *"subir los cambios" se llama hacer push y "traer los cambios de los demás" se llama hacer pull. Con solo entender esas dos palabras + el commit, ya tienes lo esencial.*

10.1 El repositorio (nuestra página en GitHub)

El proyecto vive en esta dirección (es público, todos pueden verlo aunque no tengan cuenta):

```
https://github.com/chrissaraviagomez-lab/salunic-app
```

Ahí encuentras: los archivos del proyecto, el **Documento Explicativo del Código** (este documento), la carpeta de documentos del II semestre, y el **historial de commits** (cada cambio con su fecha y su mensaje).

10.2 Cómo ver el documento explicativo desde GitHub (sin instalar nada)

52. Entra a: `https://github.com/chrissaraviagomez-lab/salunic-app`
53. En la lista de archivos, dale clic a **`Documento Explicativo del Código.md`**.
    GitHub lo abre con formato bonito (títulos, tablas, código y capturas).
54. Si quieres el archivo directo (para bajar el .docx o el PDF), dale clic al archivo y luego al botón **`Download`** o **`Raw`**.

> ***Consejo:*** *para leer este documento en tu celular también funciona — el formato se adapta solo.*

10.3 Descargar todo el proyecto (la forma más fácil)

Sin instalar nada, baja el proyecto como una carpeta comprimida:

55. Abre `https://github.com/chrissaraviagomez-lab/salunic-app`

56. Clic en el botón verde **`Code`** (arriba a la derecha).
57. Clic en **`Download ZIP`**.
58. Descomprime el archivo donde quieras (ej: en tu carpeta de Documentos).

Para **ejecutar la app** después de bajar el ZIP, abre una terminal dentro de la carpeta descomprimida y escribe `py main.py`. (Necesitas Python instalado; el ZIP ya trae todo el código.)

10.4 Instalar Git (si quieres trabajar con los comandos)

La forma *de programador* es instalar Git y clonar el proyecto. Con Git cada integrante puede actualizarlo sin volver a bajar el ZIP:

59. Descarga e instala Git desde: `https://git-scm.com/download/win` (siguiente → siguiente → instalar, con las opciones por defecto).
60. Abre la terminal (CMD o PowerShell) y escribe:

```
git clone https://github.com/chrissaraviagomez-lab/salunic-app.git
```

Aparecerá una carpeta llamada `salunic-app` con todo el proyecto. Eso se llama **clonar** (descargar una copia completa que se puede sincronizar).

10.5 Los 5 comandos que todos debemos conocer

Comando	Qué hace	Cuándo lo uso
git status	Muestra qué archivos cambiaron y cuáles faltan por subir.	Cuando quiero ver "qué hay pendiente".
git add .	"Marca" todos los cambios para guardarlos.	Antes de hacer un commit.
git commit -m "mensaje"	Guarda una "foto" (commit) de los cambios con su descripción.	Cuando terminé un cambio y quiero registrarlo.
git push	Sube los commits al GitHub (la nube).	Para que los demás vean mis cambios.
git pull	Trae los cambios que otros subieron al GitHub.	Antes de empezar a trabajar y siempre que alguien avise que subió algo.

Ejemplo de un flujo completo de trabajo:

```
git pull                                # 1. traigo lo más reciente
# ... hago mis cambios a los archivos ...
git status                              # 2. reviso qué cambié
git add .                               # 3. marco los cambios
git commit -m "Actualice el documento"  # 4. guardo la foto con su mensaje
git push                                # 5. lo subo al GitHub
```

Reglas para no romper nada:

> *1. **Siempre haz `git pull`** antes de empezar a editar.*
>
> *2. No borres archivos que no conozcas.*
>
> *3. Escribe mensajes de commit **claros y en español** (ej: "Agrego validación del email").*
>
> *4. Si algo "no se ve bien", avisa al coordinador (Cristian) antes de forzar cambios.*
>
> *5. Los archivos `.docx`, `.md` y el PDF no deben abrirse a la vez que otros: si dos personas editan lo mismo al mismo tiempo, habrá conflictos.*

10.6 Cómo actualizar "el documento en físico" (el PDF)

El PDF vive en la carpeta local `Documento Explicativo del Código/`. Cuando alguien actualice el documento fuente (`.md`), solo hay que:

61. Hacer `git pull` para traer el `.md` más nuevo.
62. Abrir la terminal en la raíz del proyecto.
63. Ejecutar:

```
py scripts\actualizar_documento.py
```

Esto regenera el Word (`.docx`) y el PDF automáticamente. Después ya se puede imprimir o leer la versión nueva. *(Requiere tener instalado Word para que se genere el PDF.)*
